

CSE-719 (Distributed and Cloud Computing)

Dr. Atiqur Rahman

ড. আতিকুর রহমান

Ph.D.(CQUPT, China), MS.Engg.(CU), B.Sc.(CU)

Associate Professor

Department of Computer Science and Engineering

University of Chittagong

Lecture 4: Failure Detection and Membership

A Challenge

- You've been put in charge of a datacenter, and your manager has told you, "Oh no! We don't have any failures in our datacenter!"
- Do you believe him/her?
- What would be your first responsibility?
- Build a failure detector
- What are some things that could go wrong if you didn't do this?

Failures are the Norm

... not the exception, in datacenters.

Say, the rate of failure of one machine (OS/disk/motherboard/network, etc.) is once every 10 years (120 months) on average.

When you have 120 servers in the DC, the **mean time to failure (MTTF)** of the next machine is 1 month.

When you have 12,000 servers in the DC, the MTTF is about once every 7.2 hours!

Soft crashes and failures are even more frequent!

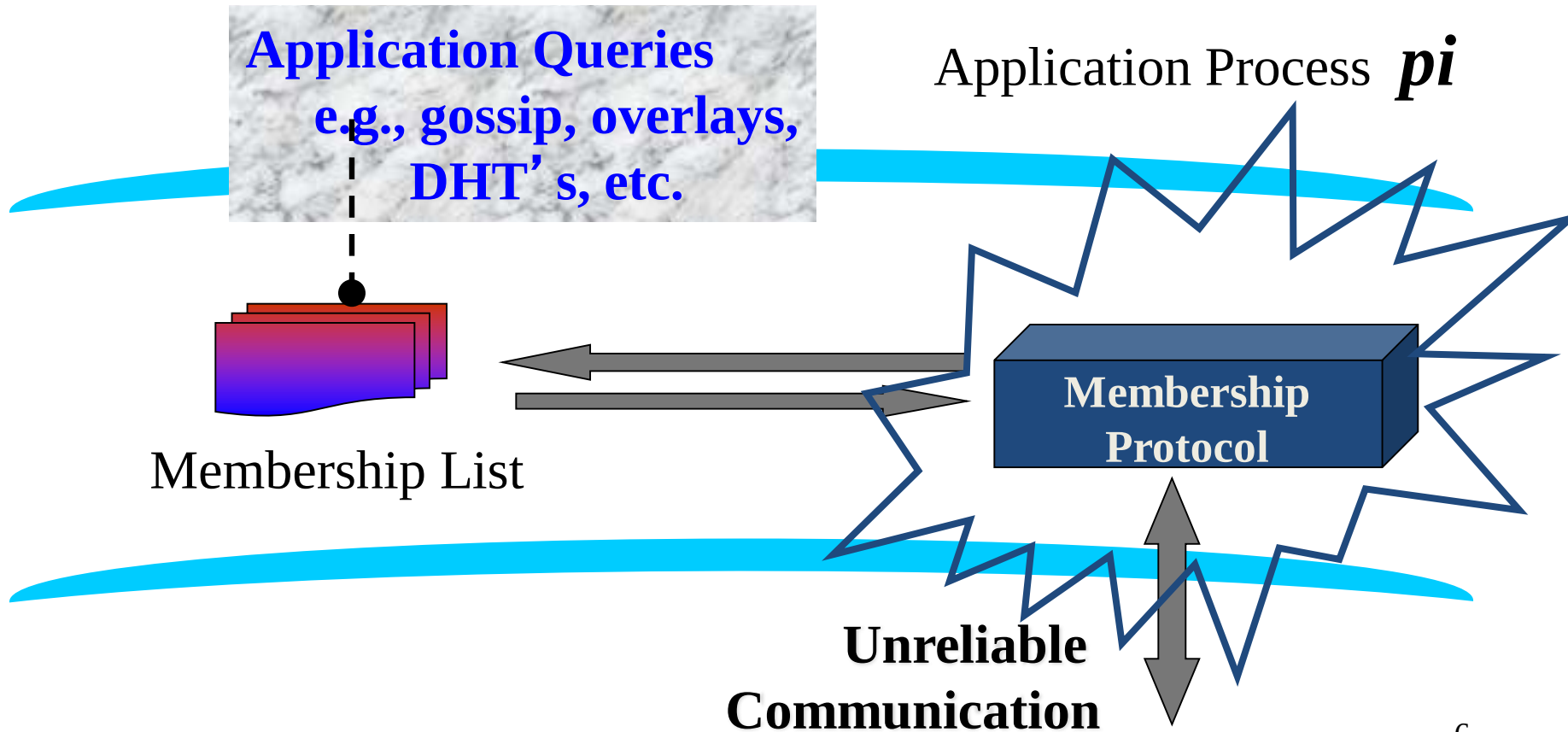
To build a failure detector

- You have a few options
 1. Hire 1000 people, each to monitor one machine in the datacenter and report to you when it fails.
 2. Write a failure detector program (distributed) that automatically detects failures and reports to your workstation.

Target Settings

- Process ‘group’ -based systems
 - Clouds/Datacenters
 - Replicated servers
 - Distributed databases
- Crash-stop/Fail-stop process failures

Group Membership Service

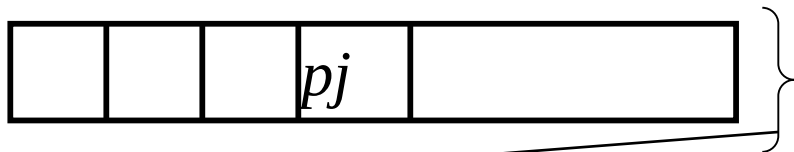


Two sub-protocols

Application Process pi

Group

Membership List



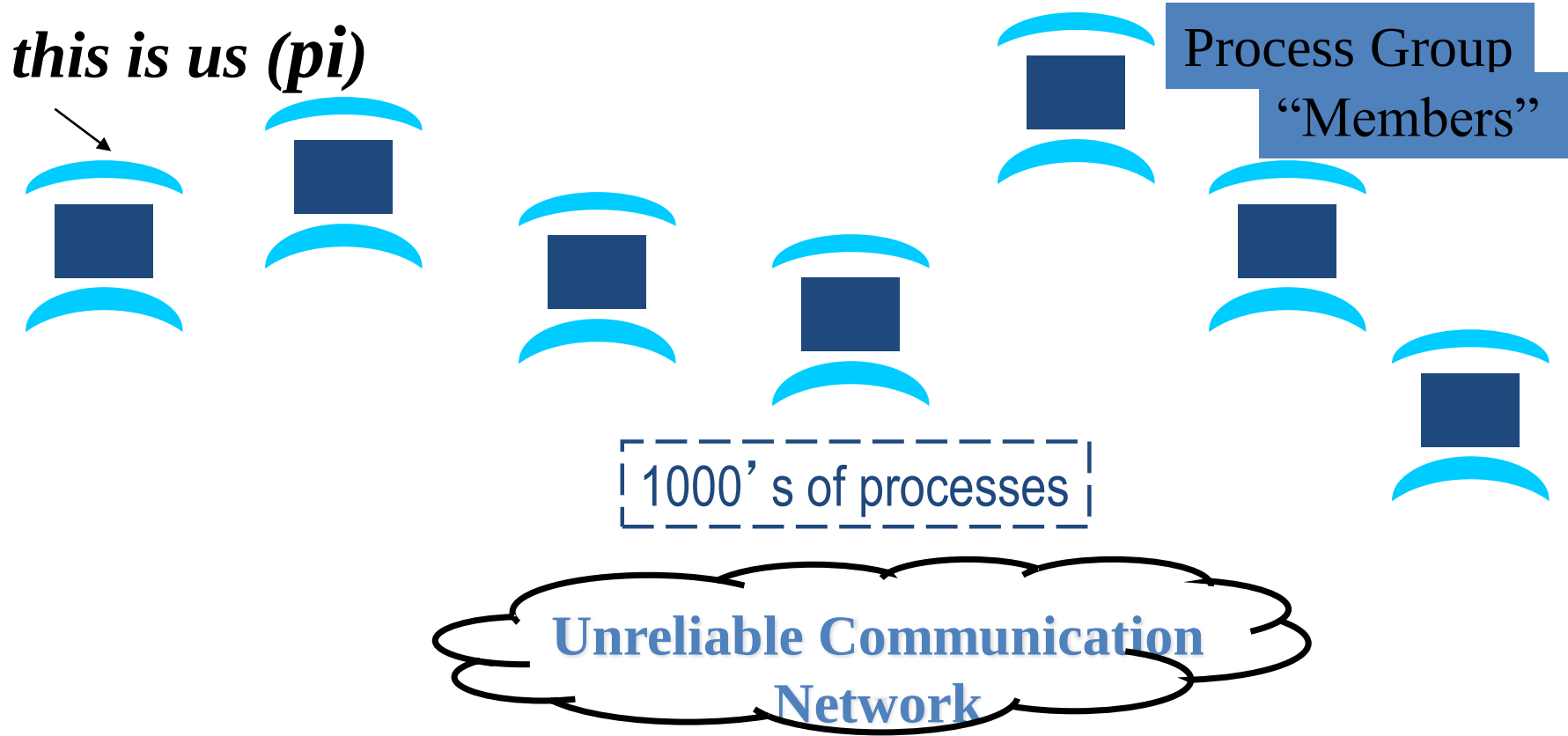
- **Complete list all the time** (Strongly consistent)
 - Virtual synchrony
- **Almost-Complete list** (Weakly consistent)
 - Gossip-style, SWIM, ...
- Or **Partial-random list** (other systems)
 - SCAMP, T-MAN, Cyclon, ...



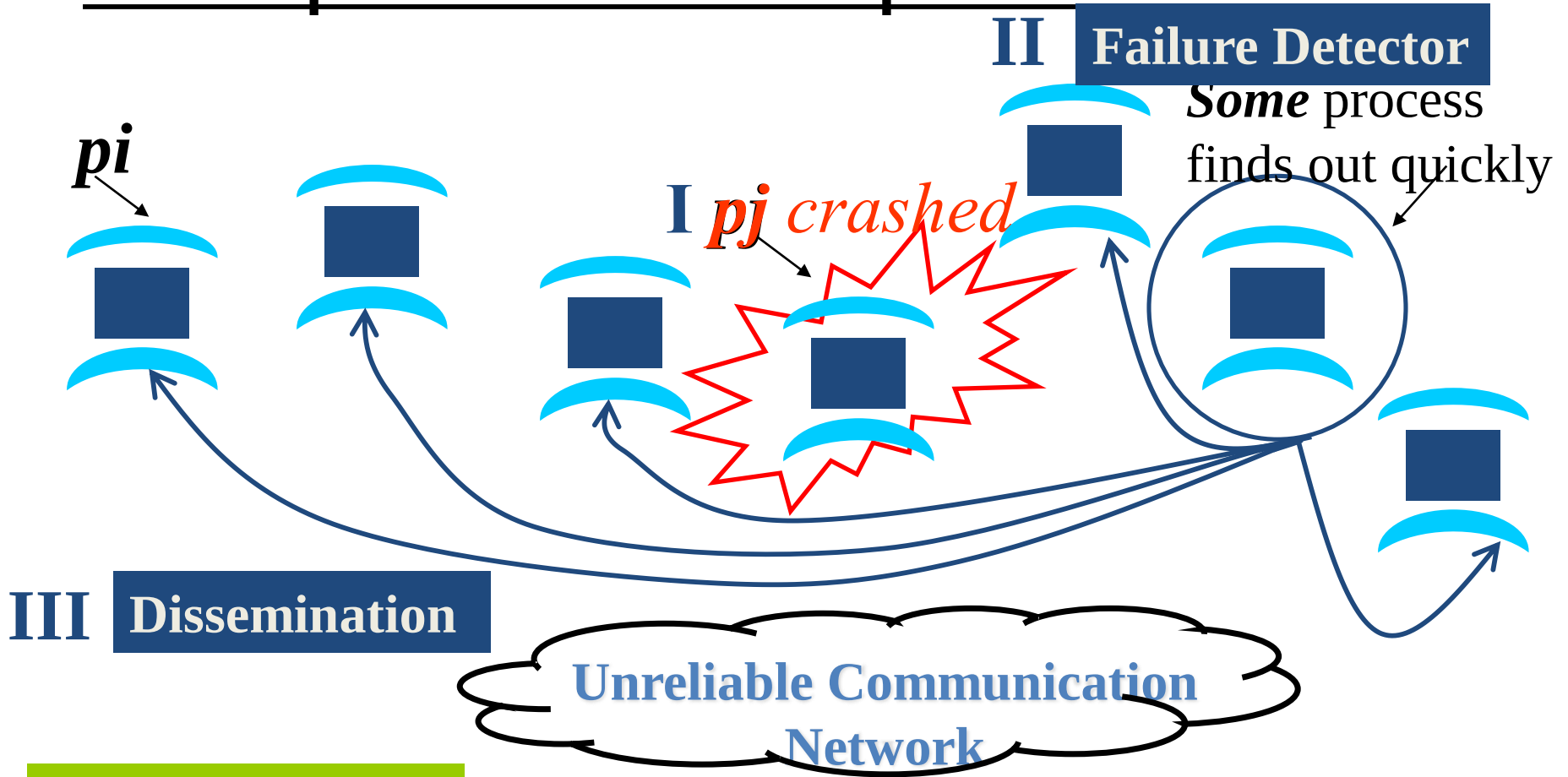
Focus of this series of lecture

Unreliable
Communication

Large Group: Scalability A Goal



Group Membership Protocol



Crash-stop Failures only

Next

- How do you design a group membership protocol?

I. *pj* crashes

- Nothing we can do about it!
- A frequent occurrence
- Common case rather than exception
- Frequency goes up linearly with size of datacenter

II. Distributed Failure Detectors: Desirable Properties

- **Completeness** = each failure is detected
- **Accuracy** = there is no mistaken detection
- Speed
 - Time to first detection of a failure
- Scale
 - Equal Load on each member
 - Network Message Load

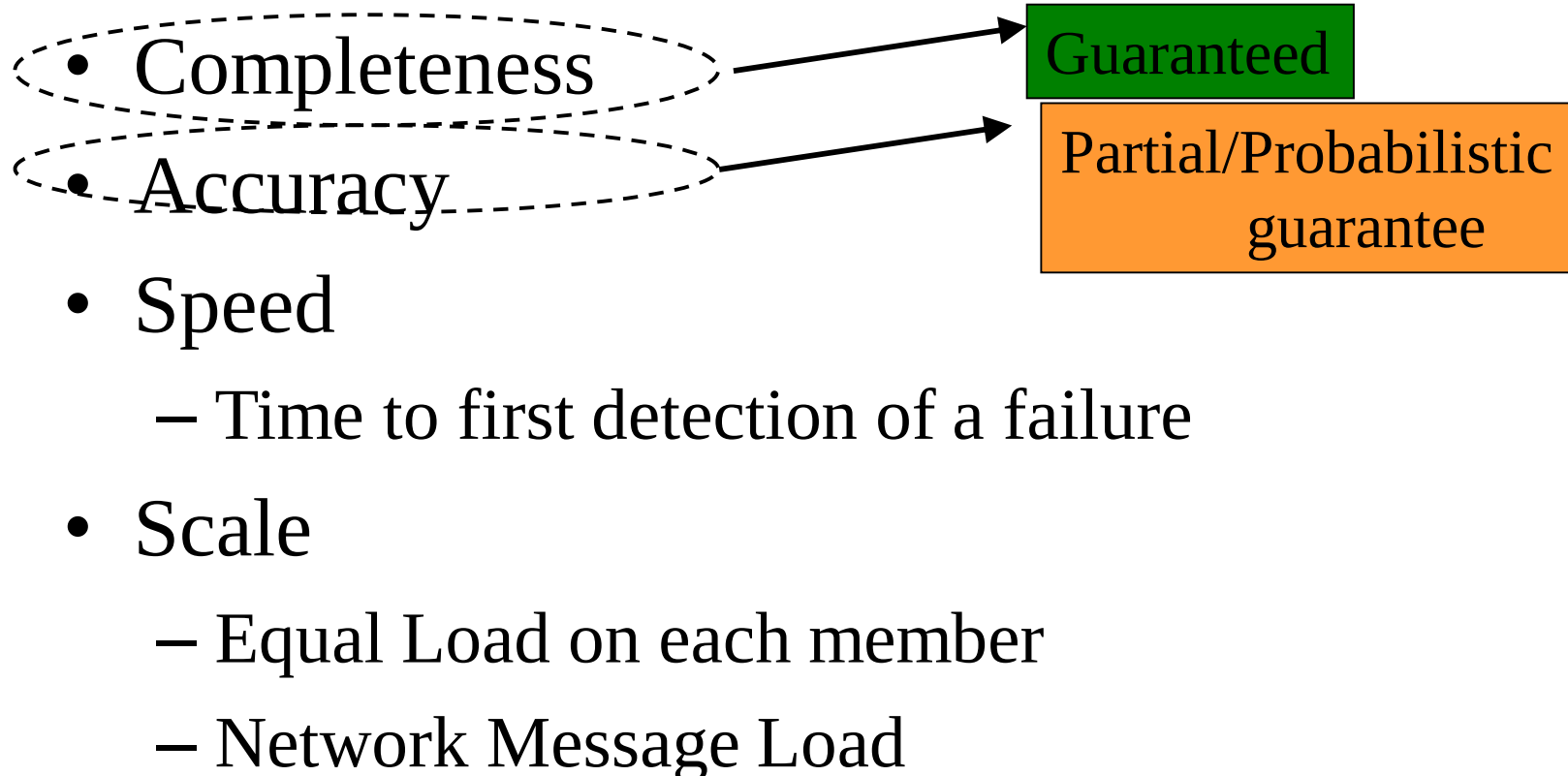
Distributed Failure Detectors: Properties

- Completeness
- Accuracy
- Speed
 - Time to first detection of a failure
- Scale
 - Equal Load on each member
 - Network Message Load

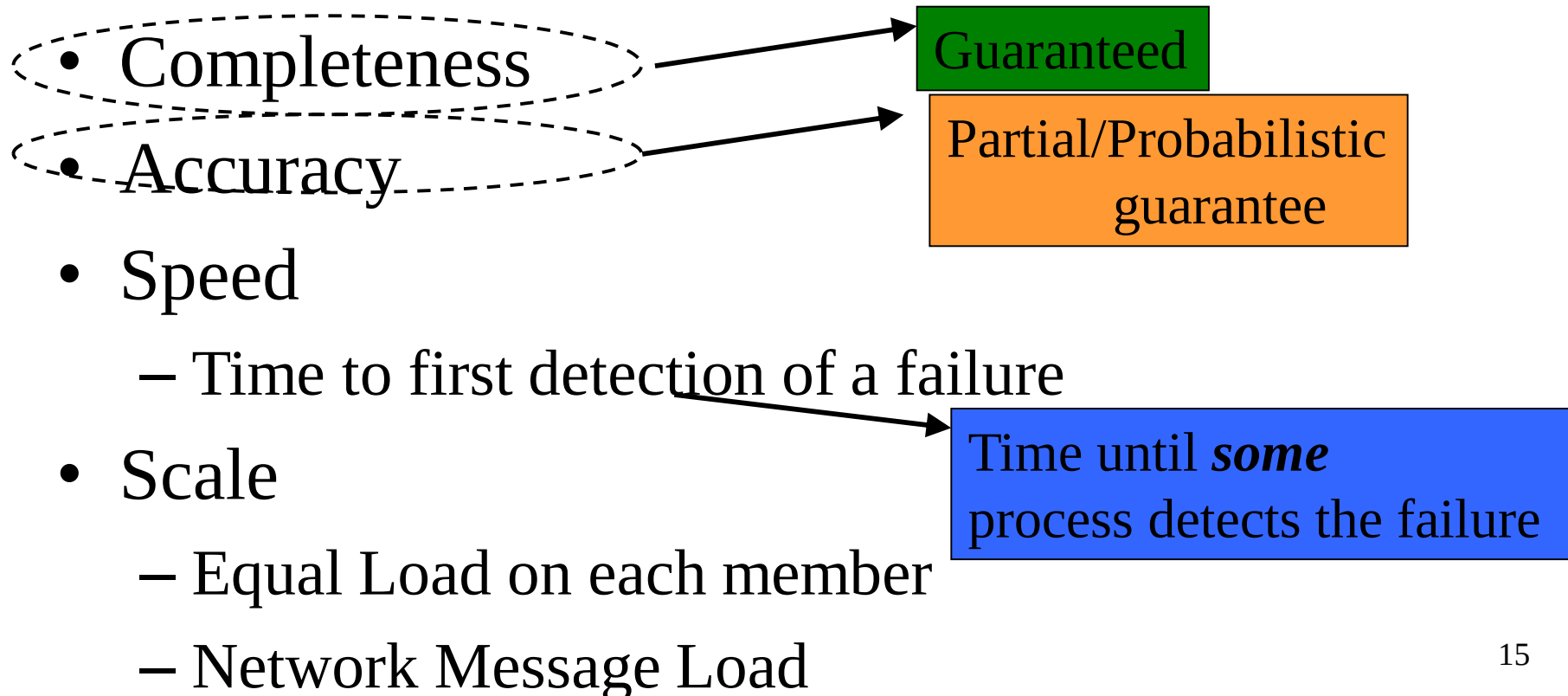
Impossible together in lossy networks [Chandra and Toueg]

If possible, then can solve consensus!

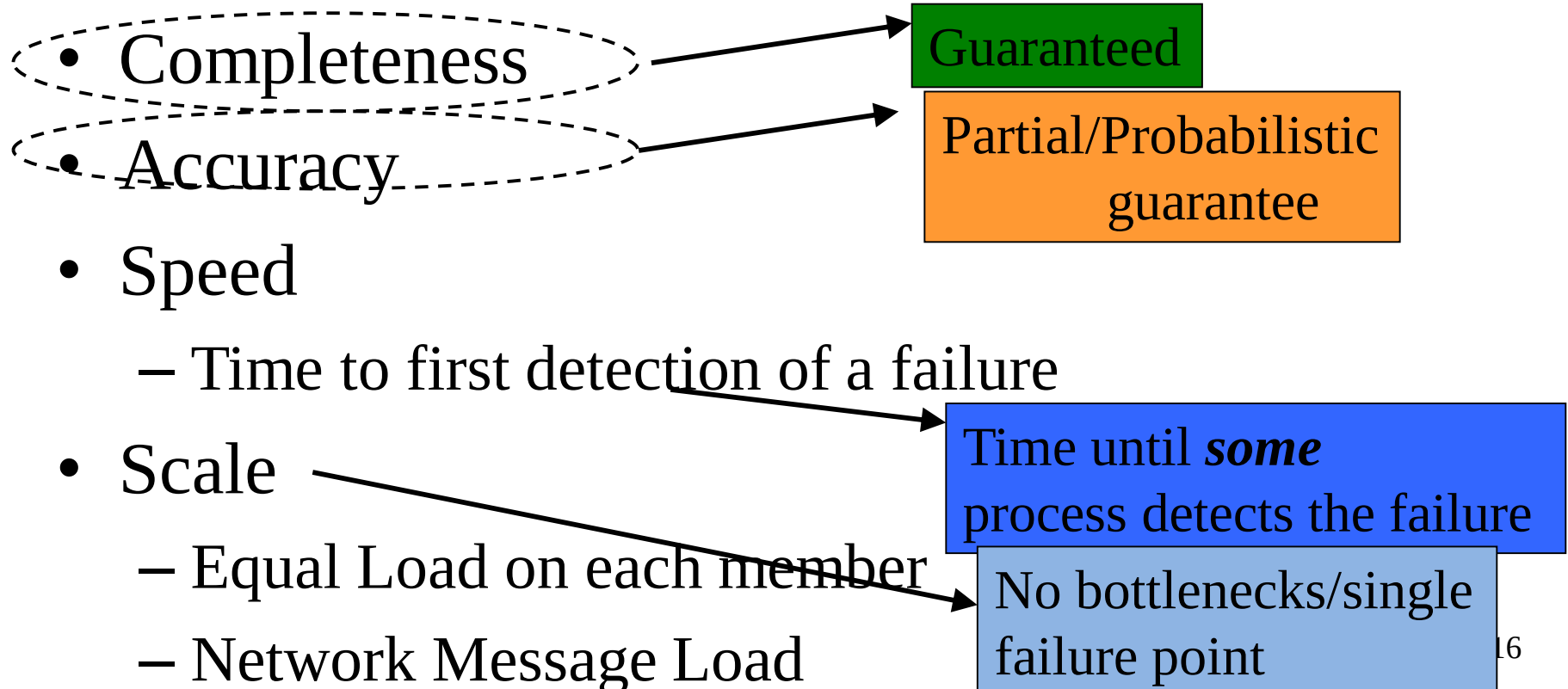
What Real Failure Detectors Prefer



What Real Failure Detectors Prefer



What Real Failure Detectors Prefer

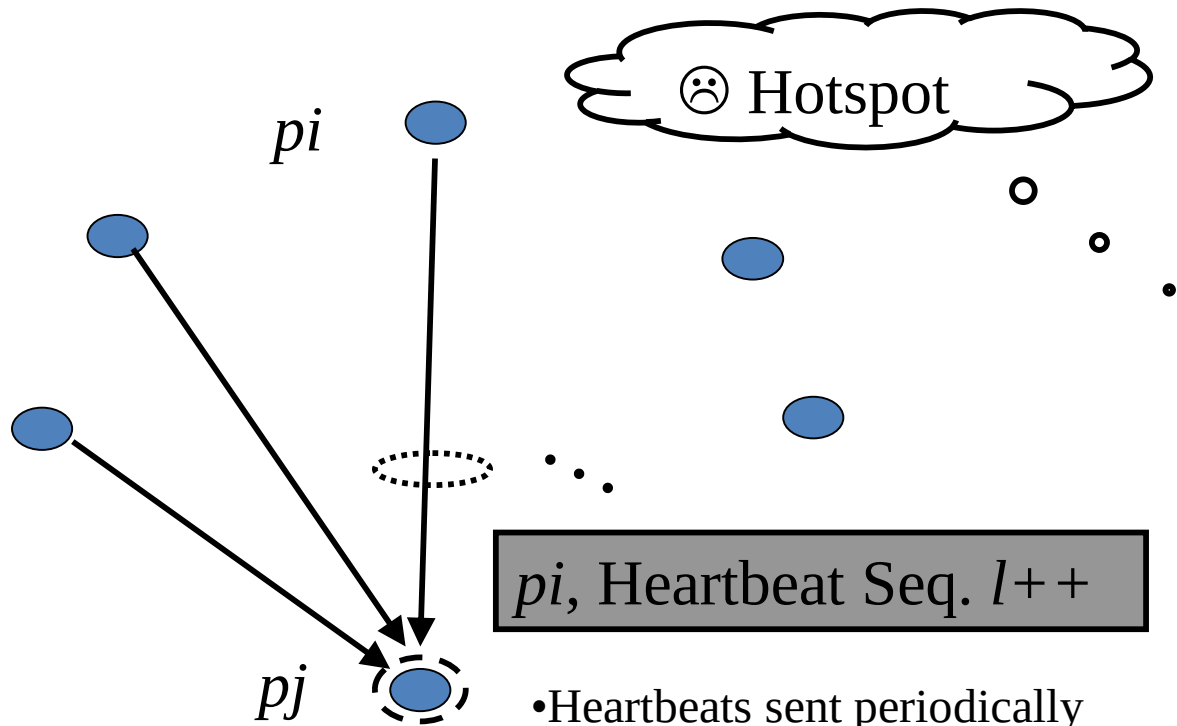


Failure Detector Properties

- Completeness
- Accuracy
- Speed
 - Time to first detection of a failure
- Scale
 - Equal Load on each member
 - Network Message Load

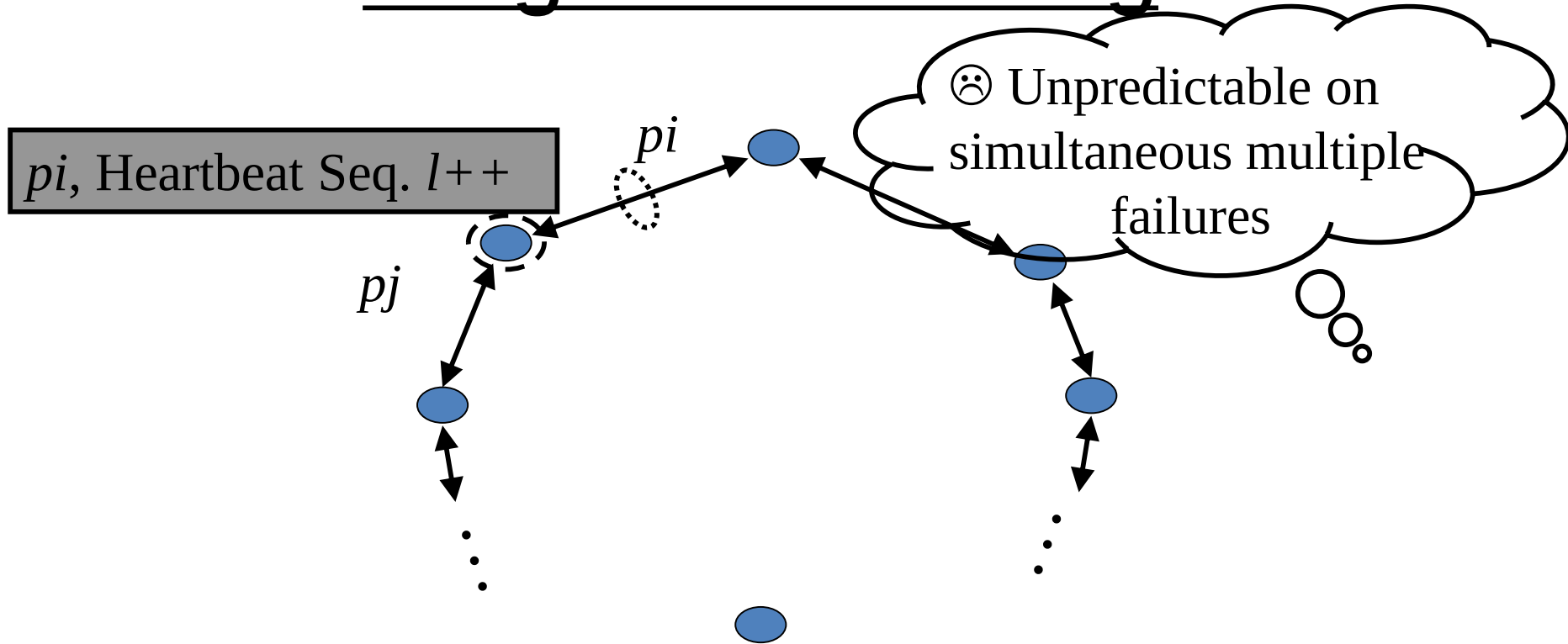
In spite of
arbitrary simultaneous
process failures

Centralized Heartbeating

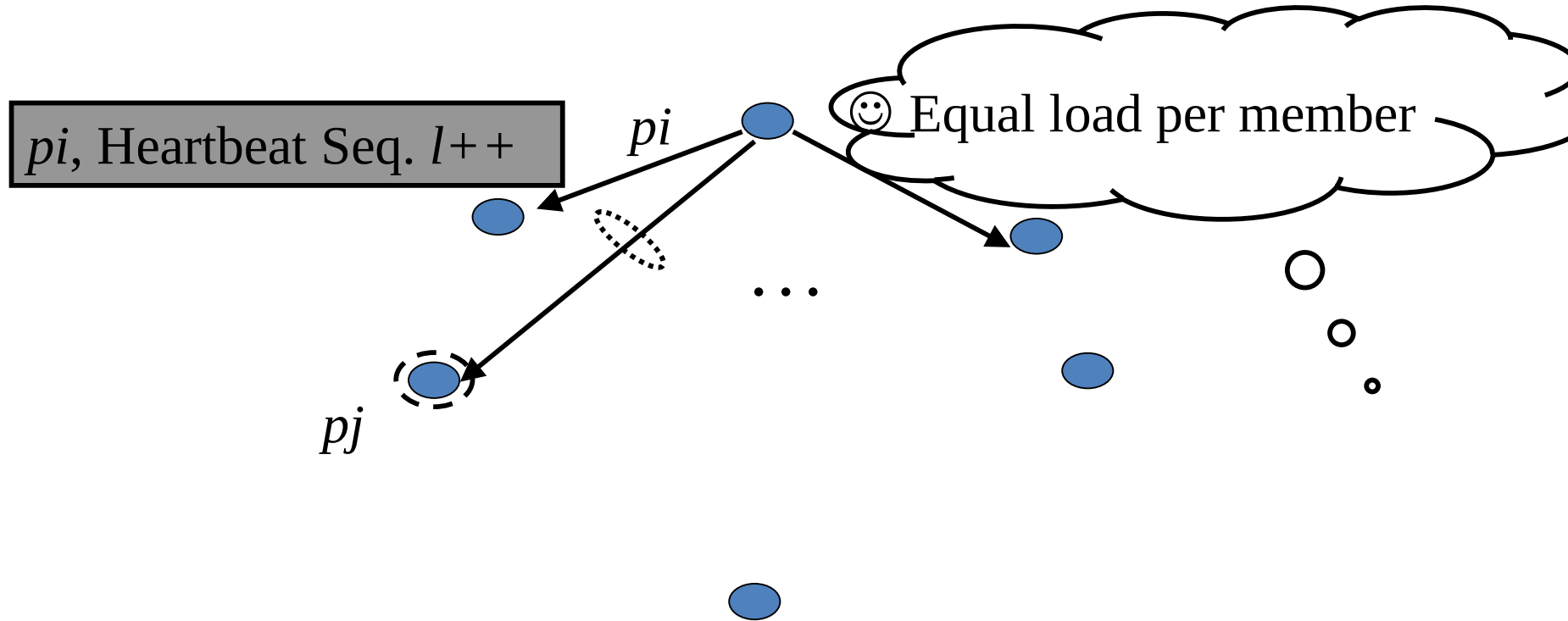


- Heartbeats sent periodically
- If heartbeat not received from p_i within timeout, mark p_i as failed

Ring Heartbeating



All-to-All Heartbeating

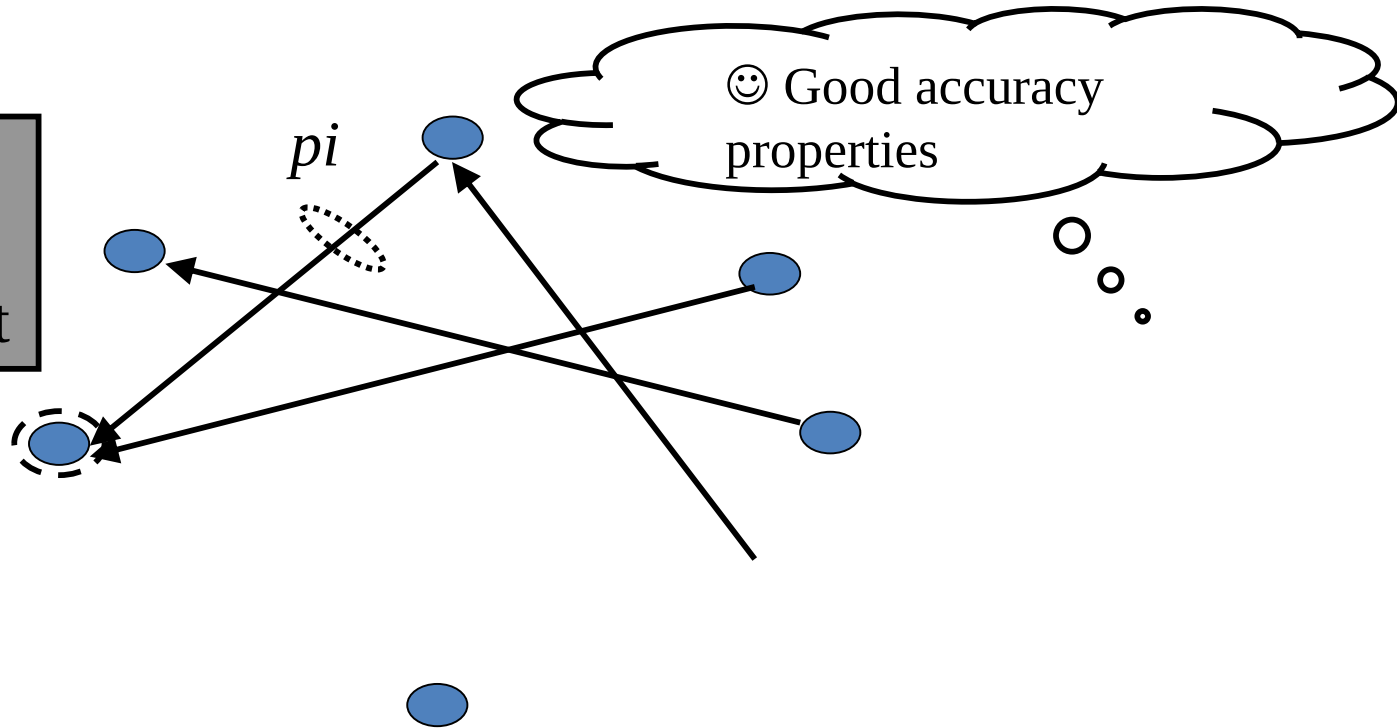


Next

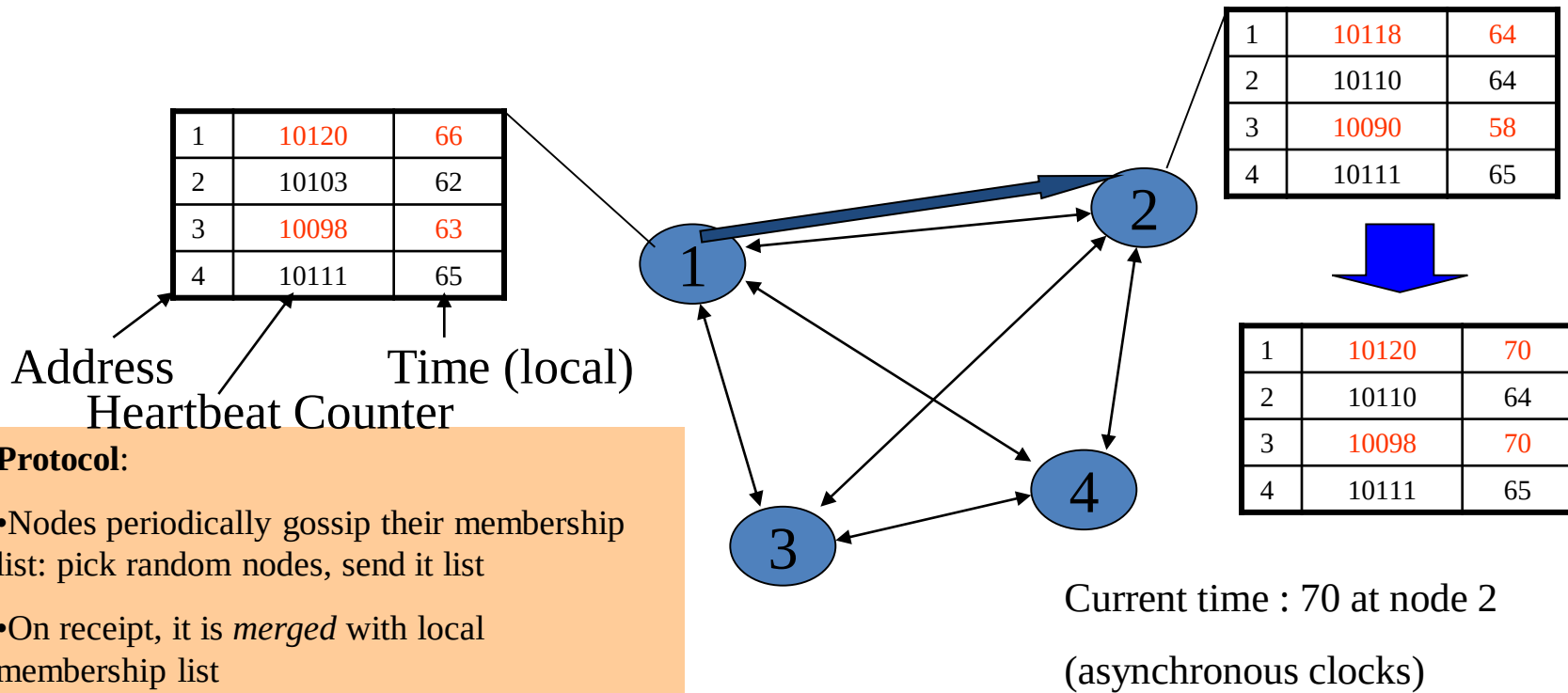
- How do we increase the robustness of all-to-all heartbeating?

Gossip-style Heartbeating

Array of
Heartbeat Seq. l
for member subset



Gossip-Style Failure Detection



Protocol:

- Nodes periodically gossip their membership list: pick random nodes, send it list
- On receipt, it is *merged* with local membership list
- When an entry times out, member is marked as failed

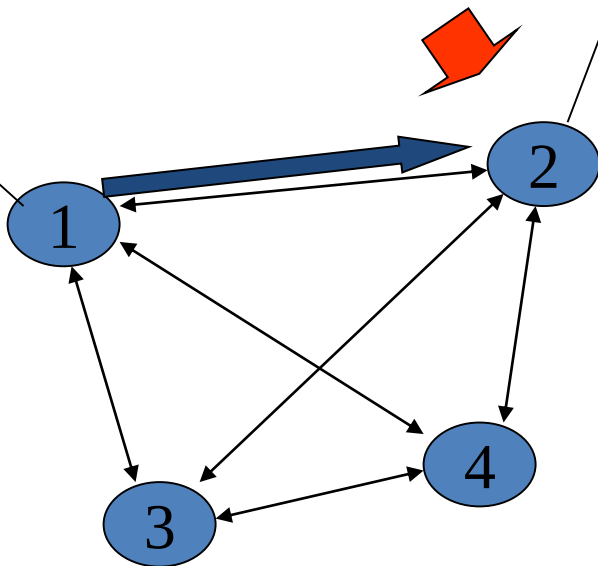
Gossip-Style Failure Detection

- If the heartbeat has not increased for more than T_{fail} seconds,
the member is considered failed
- And after T_{cleanup} seconds, it will delete the member from the list
- Why two different timeouts?

Gossip-Style Failure Detection

- What if an entry pointing to a failed node is deleted right after T_{fail} ($=24$) seconds?

1	10120	66
2	10103	62
3	10098	55
4	10111	65



1	10120	66
2	10110	64
3	10098	55
4	10111	65

Current time : 75 at node 2

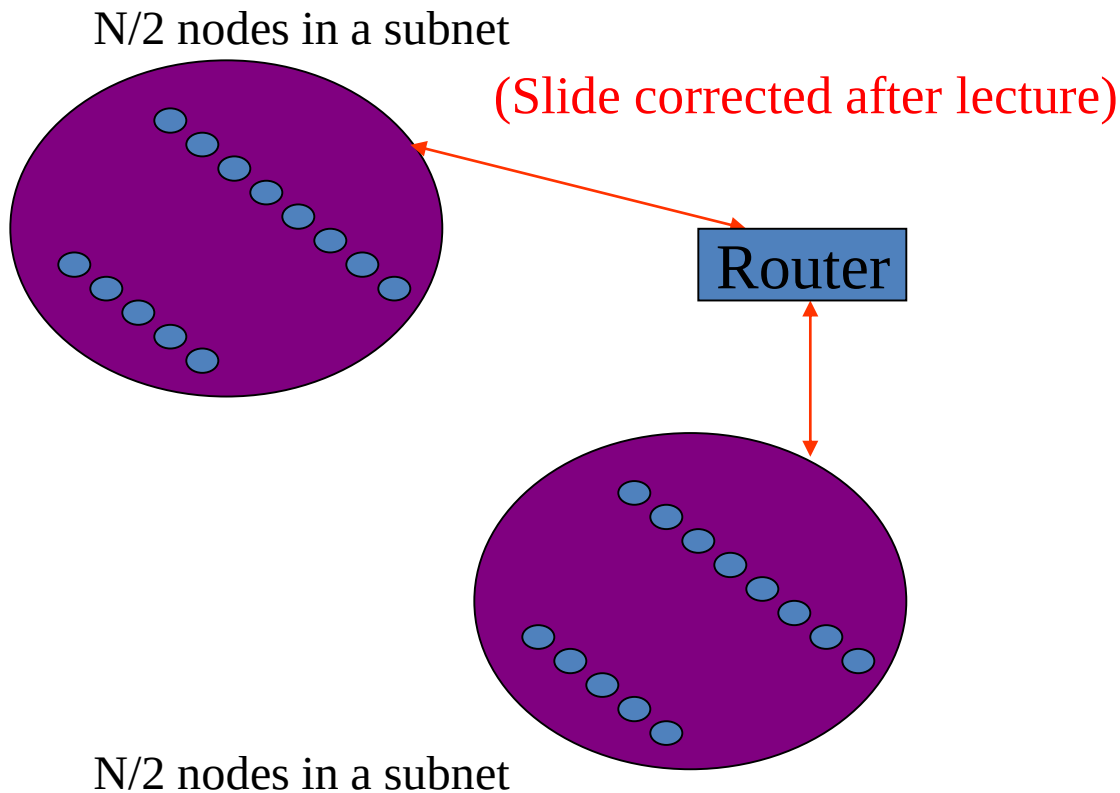
Multi-level Gossiping

- Network topology is hierarchical
- Random gossip target selection
=> core routers face $O(N)$ load
(Why?)

• **Fix:** In subnet i , which contains n_i nodes, pick gossip target in your subnet with probability $(1 - 1/n_i)$

- Router load = $O(1)$
- Dissemination time = $O(\log(N))$
- What about latency for multi-level topologies?

[Gupta et al, TPDS 06]



Analysis/Discussion

- What happens if gossip period T_{gossip} is decreased?
- A single heartbeat takes $O(\log(N))$ time to propagate. So: N heartbeats take:
 - $O(\log(N))$ time to propagate, if bandwidth allowed per node is allowed to be $O(N)$
 - $O(N \cdot \log(N))$ time to propagate, if bandwidth allowed per node is only $O(1)$
 - What about $O(k)$ bandwidth?
- What happens to P_{mistake} (false positive rate) as $T_{\text{fail}}, T_{\text{cleanup}}$ is increased?
- Tradeoff: False positive rate vs. detection time vs. bandwidth

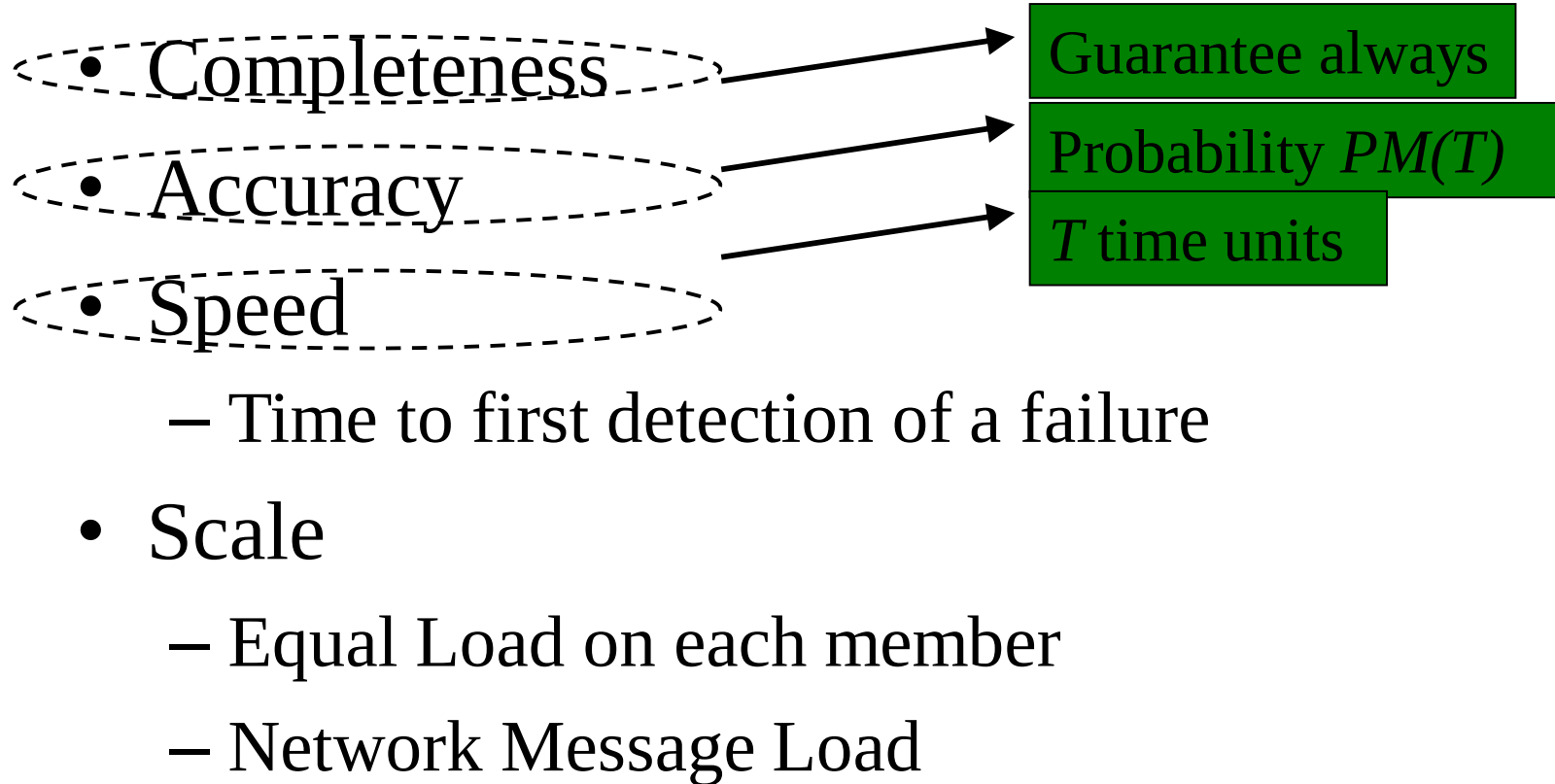
Next

- So, is this the best we can do? What is the best we can do?

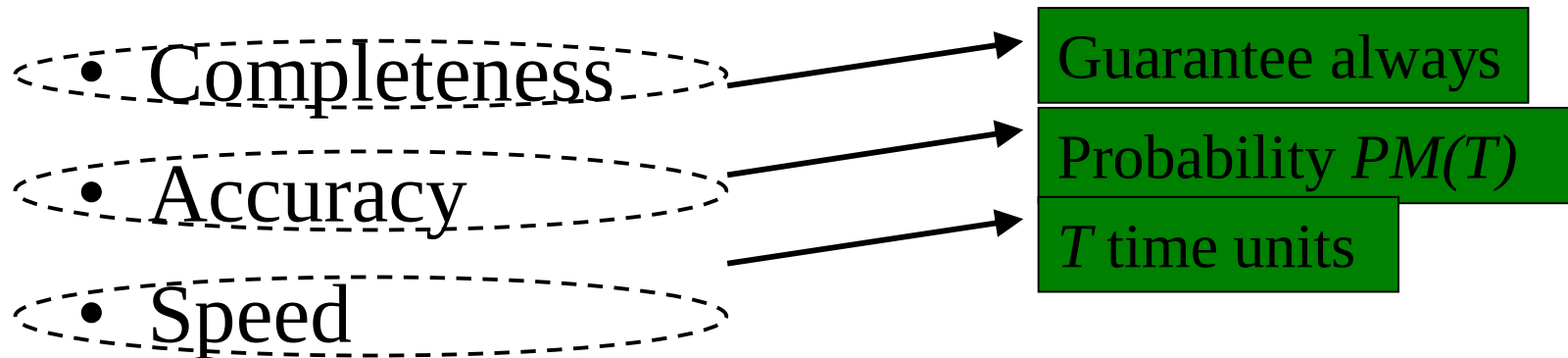
Failure Detector Properties ...

- Completeness
- Accuracy
- Speed
 - Time to first detection of a failure
- Scale
 - Equal Load on each member
 - Network Message Load

...Are application-defined Requirements



...Are application-defined Requirements



– Time to first detection of a failure

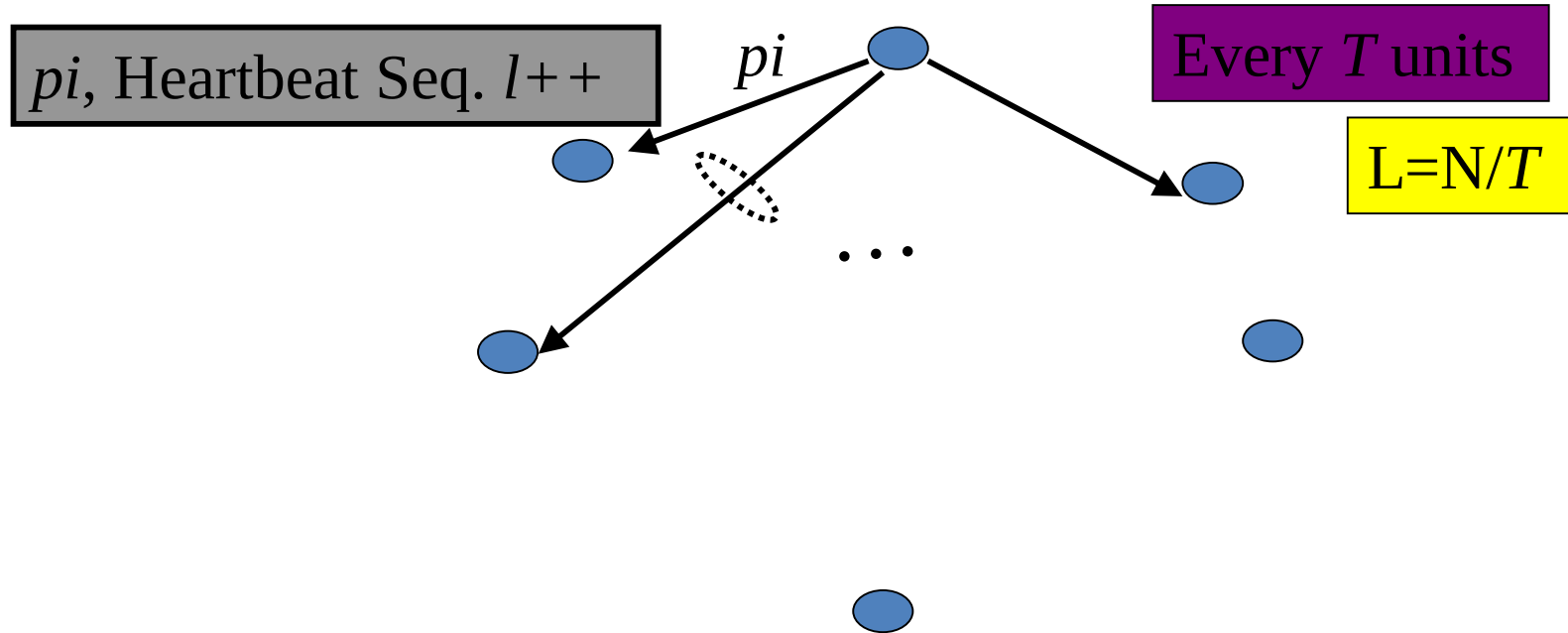
N*L: Compare this across protocols

- Scale

- Equal Load on each member

- Network Message Load

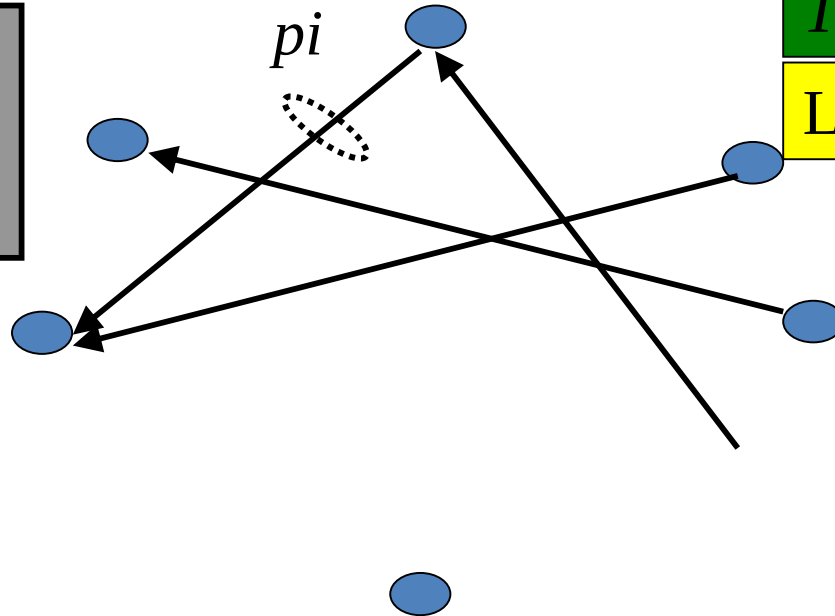
All-to-All Heartbeating



Gossip-style Heartbeating

Array of
Heartbeat Seq. l
for member subset

Every tg units
=gossip period,
send $O(N)$ gossip
message



$$T = \log N * tg$$

$$L = N/tg = N * \log N / T$$

What's the Best/Optimal we can do?

Slide changed after lecture

- *Worst case* load L^* **per member** in the group (messages per second)
 - as a function of T , $PM(T)$, N
 - Independent Message Loss probability p_{ml}

- $$L^* = \frac{\log(PM(T))}{\log(p_{ml})} \cdot \frac{1}{T}$$

Heartbeating

- Optimal L is independent of N (!)
- All-to-all and gossip-based: sub-optimal
 - $L=O(N/T)$
 - try to achieve simultaneous detection at ***all*** processes
 - fail to distinguish *Failure Detection* and *Dissemination* components

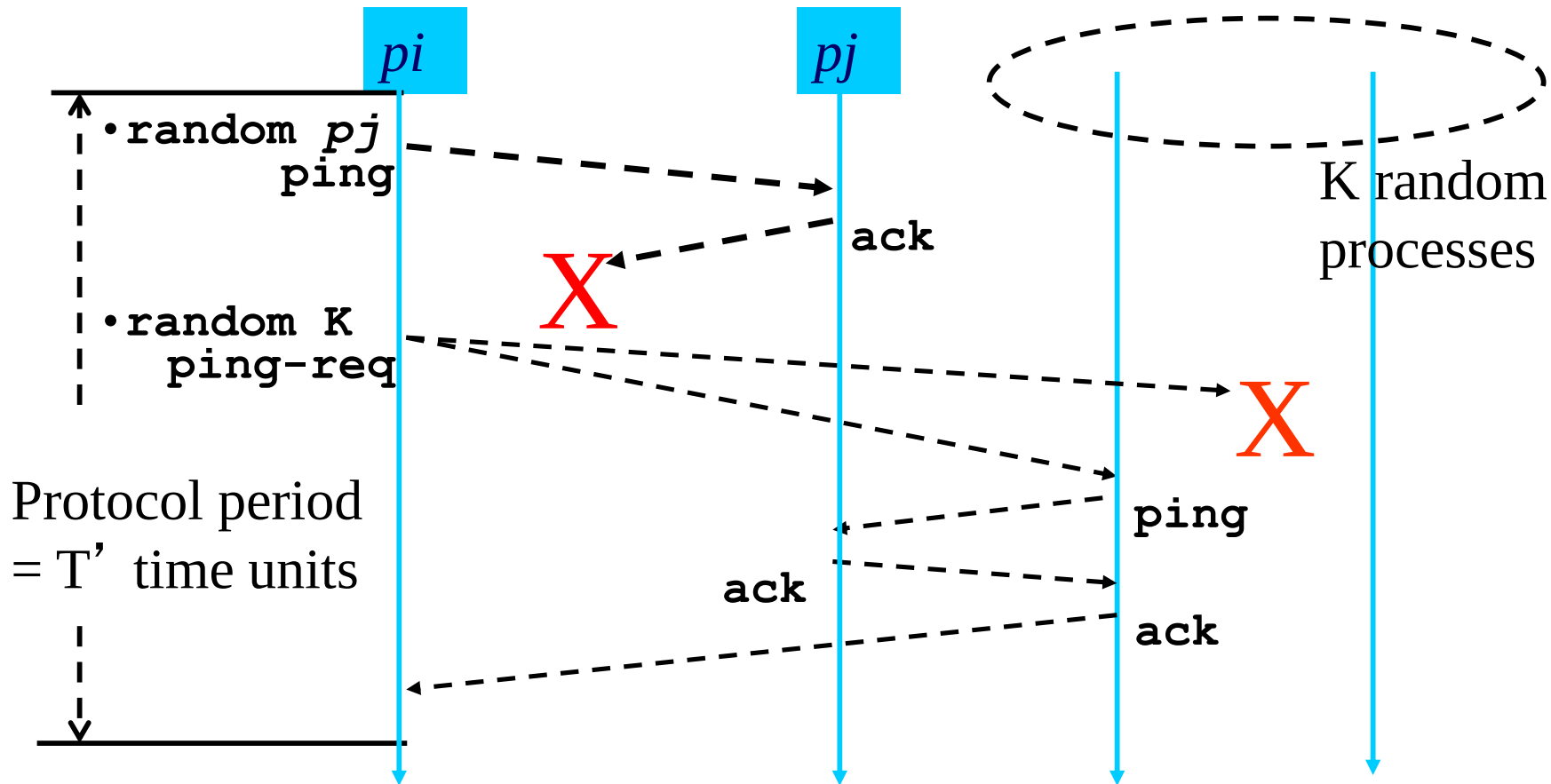
⇒ Key:

- Separate the two components
- Use a non heartbeat-based Failure Detection Component

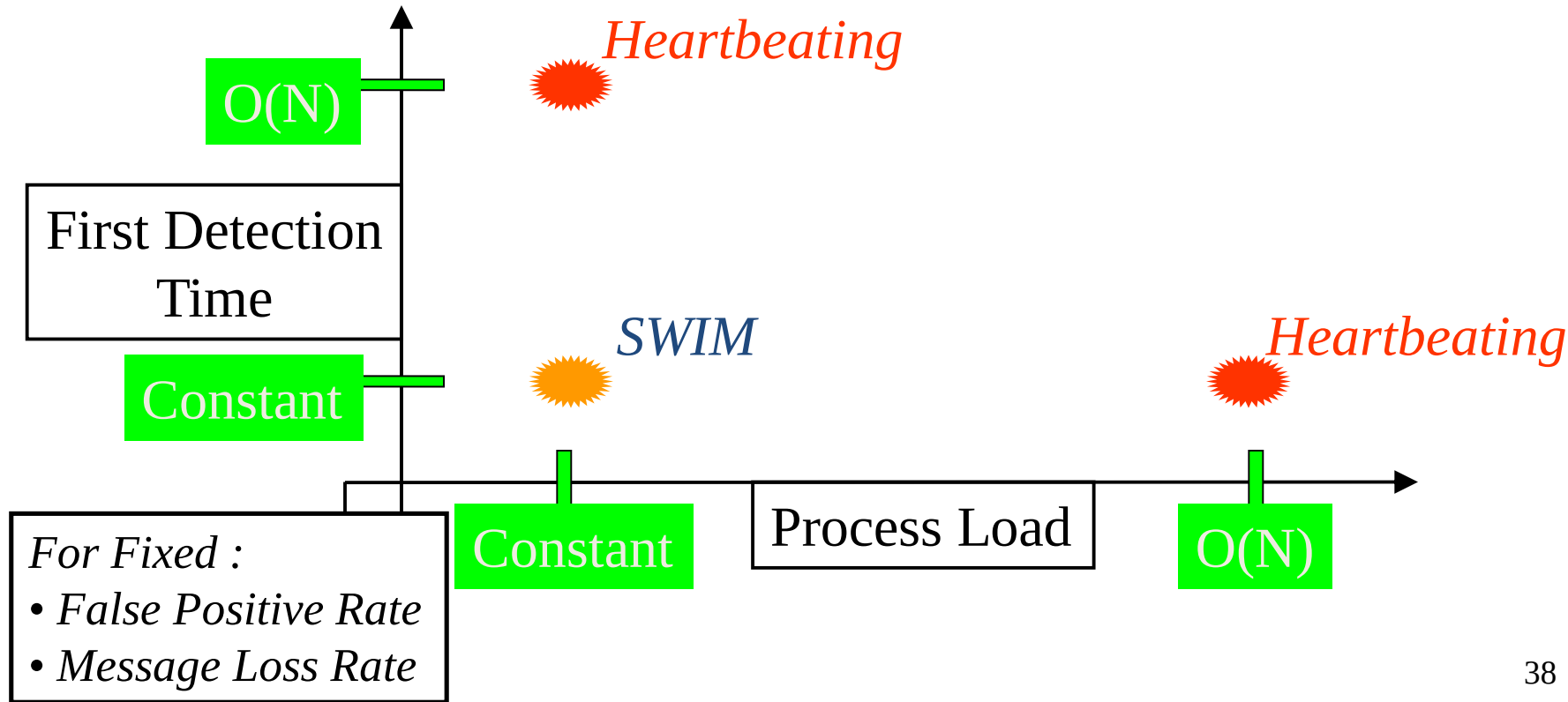
Next

- Is there a better failure detector?

SWIM Failure Detector Protocol



SWIM versus Heartbeating



SWIM Failure Detector

Parameter	SWIM
First Detection Time	• Expected $\left[\frac{e}{e-1} \right]$ periods • Constant (independent of group size)
Process Load	• Constant per period • $< 8 L^*$ for 15% loss
False Positive Rate	• Tunable (via K) • Falls exponentially as load is scaled
Completeness	• Deterministic time-bounded • Within $O(\log(N))$ periods w.h.p.

Accuracy, Load

- $PM(T)$ is exponential in $-K$. Also depends on pml (and pf)
 - See paper

- $\frac{L}{L^*} < 28$ $\frac{E[L]}{L^*} < 8$ for up to 15 % loss rates

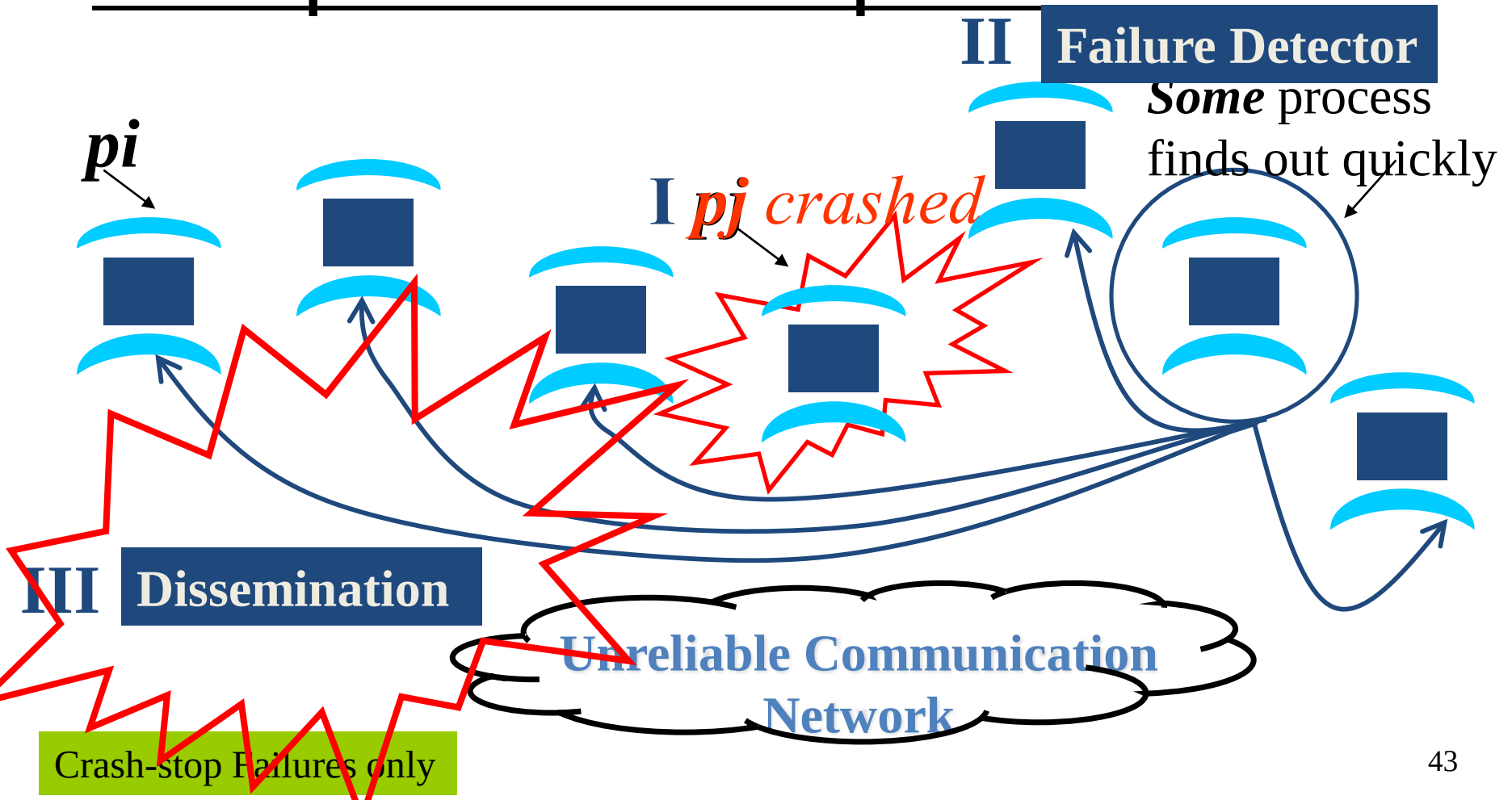
Detection Time

- Prob. of being pinged in $T' = 1 - (1 - \frac{1}{N})^{N-1} = 1 - e^{-1}$
- $E[T] = T' \cdot \frac{e}{e-1}$
- Completeness: *Any* alive member detects failure
 - Eventually
 - By using a trick: within worst case $O(N)$ protocol periods

Next

- How do failure detectors fit into the big picture of a group membership protocol?
- What are the missing blocks?

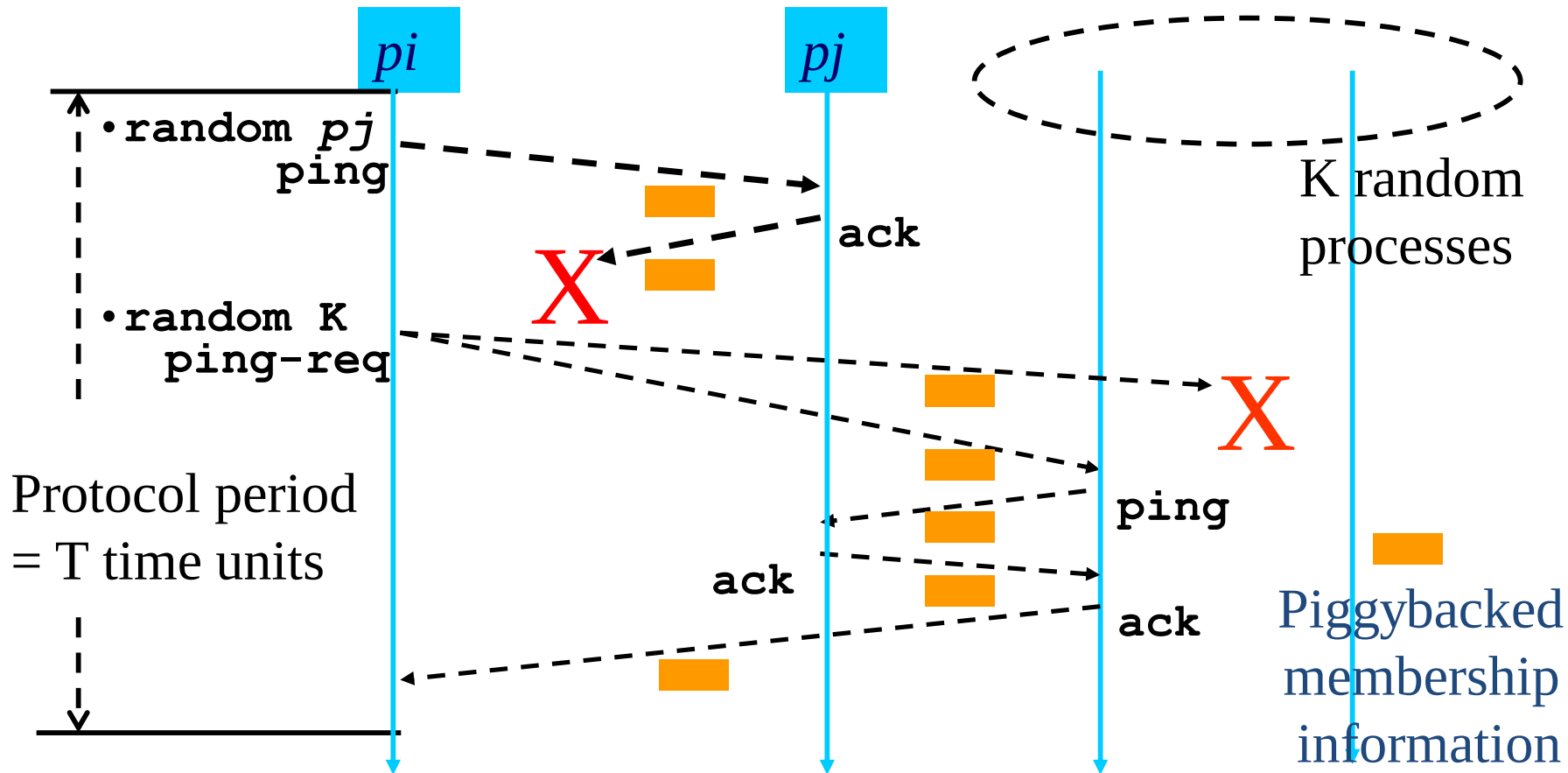
Group Membership Protocol



Dissemination Options

- Multicast (Hardware / IP)
 - unreliable
 - multiple simultaneous multicasts
- Point-to-point (TCP / UDP)
 - expensive
- Zero extra messages: Piggyback on Failure Detector messages
 - Infection-style Dissemination

Infection-style Dissemination

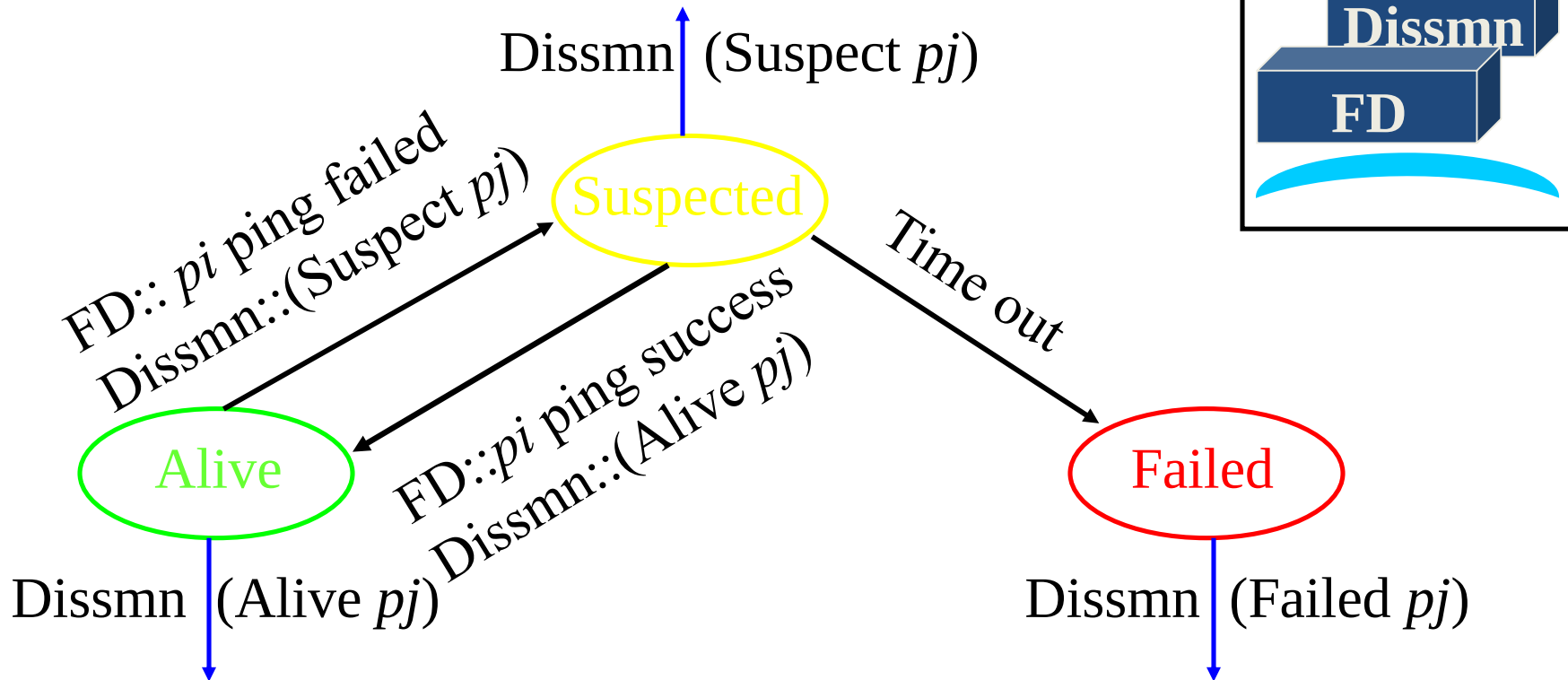


Suspicion Mechanism

- False detections, due to
 - Perturbed processes
 - Packet losses, e.g., from congestion
- Indirect pinging may not solve the problem
- Key: *suspect* a process before *declaring* it as failed in the group

Suspicion Mechanism

pi :: State Machine for pj view element



Suspicion Mechanism

- Distinguish multiple suspicions of a process
 - Per-process *incarnation number*
 - *Inc #* for p_i can be incremented only by p_i
 - e.g., when it receives a (Suspect, p_i) message
 - Somewhat similar to DSDV
- Higher inc# notifications over-ride lower inc#'s
- Within an inc#: (Suspect inc #) > (Alive, inc #)
- (Failed, inc #) overrides everything else

Wrap Up

- Failures the norm, not the exception in datacenters
- Every distributed system uses a failure detector
- Many distributed systems use a membership service

- Ring failure detection underlies
 - IBM SP2 and many other similar clusters/machines

- Gossip-style failure detection underlies
 - Amazon EC2/S3 (rumored!)